

Data Structures and Algorithms

Lecture 3: Stacks

Data Science

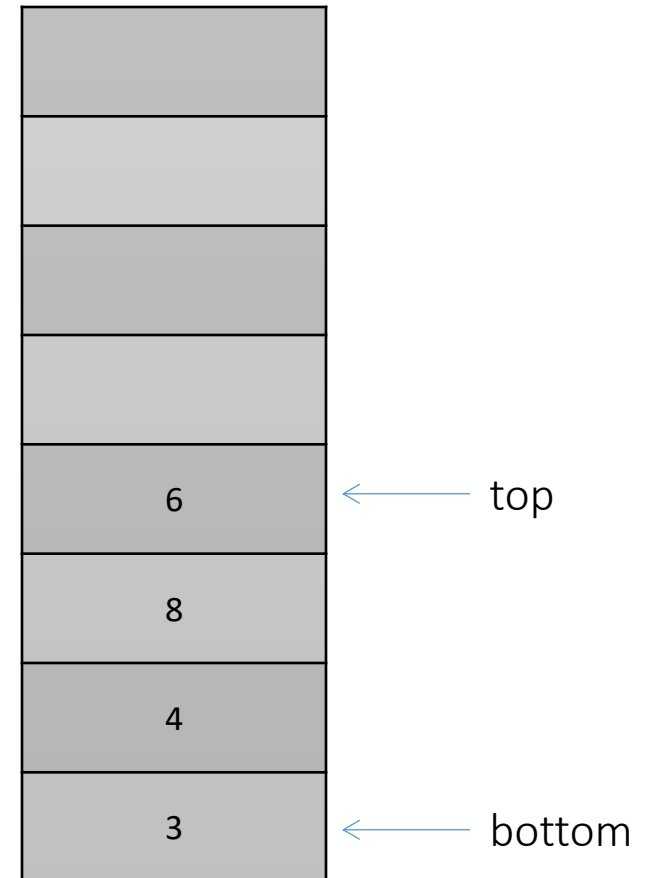
Beijing Normal-Hong Kong Baptist University

Outline

- Stack ADT
- Basic operations of stack
- Applications of stacks
- Implementations of stacks

Stack ADT

- Stack is a special list where **insertion** and **deletion** take place at the same end
 - This end is called *top*
 - The other end is called *bottom*
- **Everything** happens at the *top*
- **Nothing** happens at the *bottom*

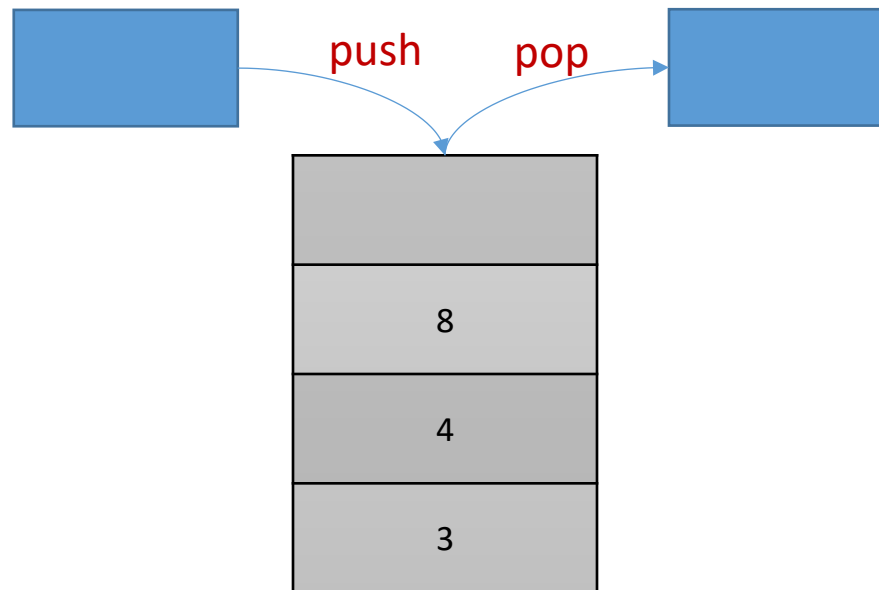


Stack Animation

- <https://www.cs.usfca.edu/~galles/visualization/StackArray.html>
- Stacks are known as **LIFO** (Last In, First Out) lists.

Push and Pop

- Primary operations: **Push** and **Pop**
 - Push : Add an element to the top of the stack
 - Pop: Remove the element at the top of the stack
- Top (or Peak)
 - Return, without removing, the element at the top

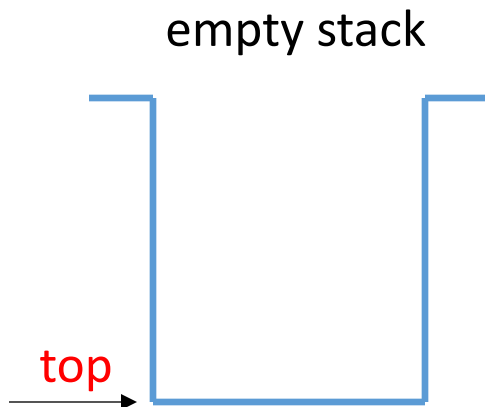


Push and Pop

- Primary operations: **Push** and **Pop**
 - Push : Add an element to the top of the stack
 - Pop: Remove the element at the top of the stack
- Top (or Peak)
 - Return, without removing, the element at the top

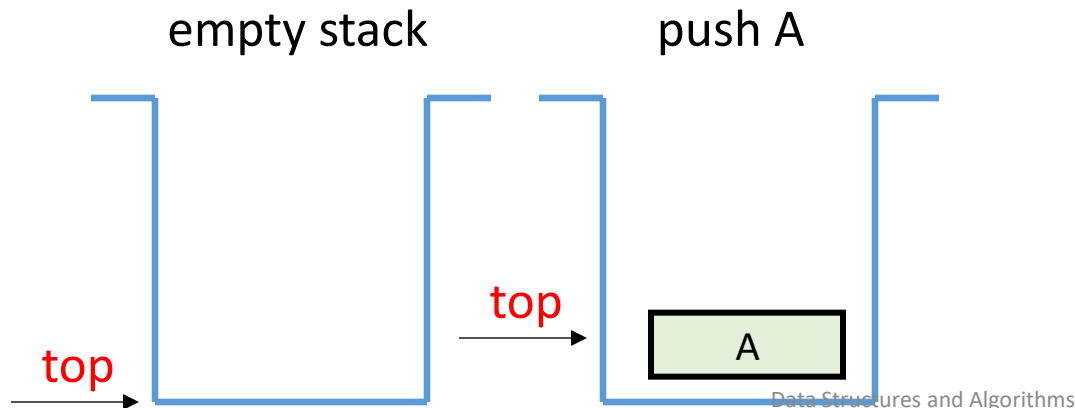
Push and Pop

- Primary operations: **Push** and **Pop**
 - Push : Add an element to the top of the stack
 - Pop: Remove the element at the top of the stack
- Top (or Peak)
 - Return, without removing, the element at the top



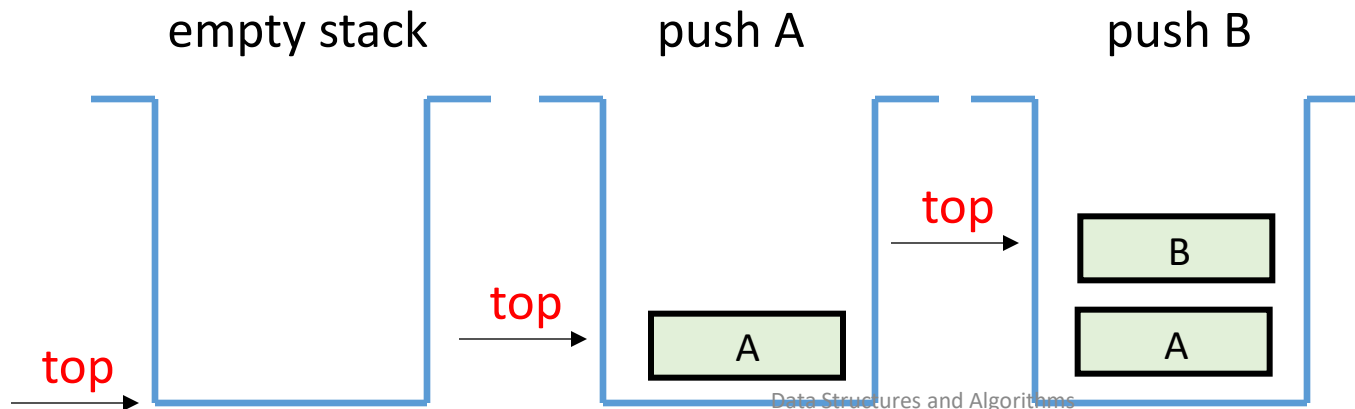
Push and Pop

- Primary operations: **Push** and **Pop**
 - Push : Add an element to the top of the stack
 - Pop: Remove the element at the top of the stack
- Top (or Peak)
 - Return, without removing, the element at the top



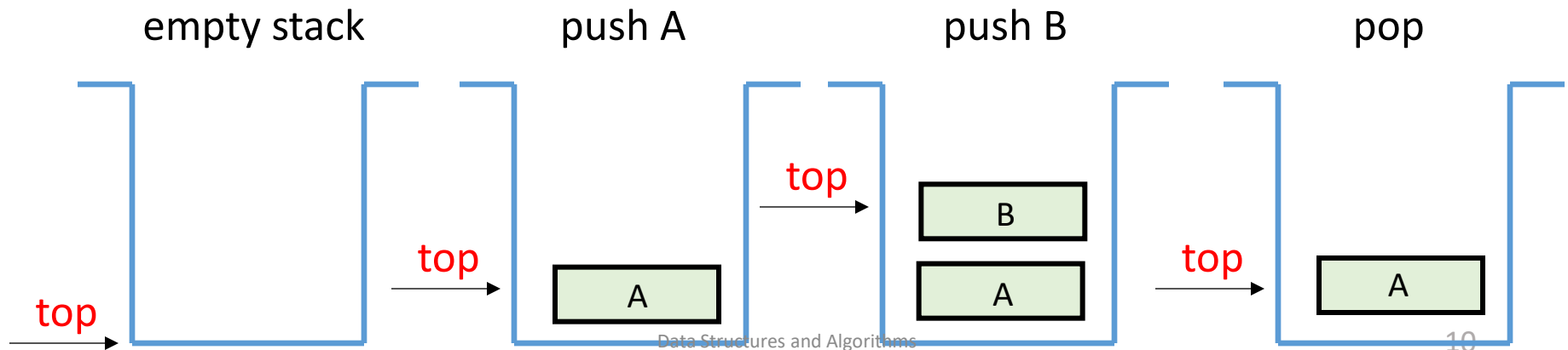
Push and Pop

- Primary operations: **Push** and **Pop**
 - Push : Add an element to the top of the stack
 - Pop: Remove the element at the top of the stack
- Top (or Peak)
 - Return, without removing, the element at the top



Push and Pop

- Primary operations: **Push** and **Pop**
 - Push : Add an element to the top of the stack
 - Pop: Remove the element at the top of the stack
- Top (or Peak)
 - Return, without removing, the element at the top



Stack Applications

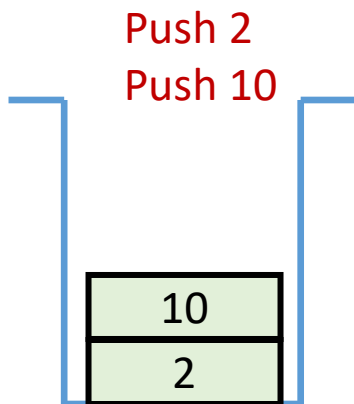
- Expression evaluation
 - Prefix, infix and postfix expression evaluation
- Backtracking
 - Game playing, finding paths, exhaustive searching
- Memory Management
 - Any modern computer environment uses a stack as the primary memory management model for a running program

Expression Evaluation

- In postfix or reverse polish notation, every operator follows all of its operands
 - E.g., 2 10 + 9 6 - /
- Algorithm for evaluation of postfix expression
 - If the element is an operand, push it to stack
 - If the element is an operator **O**, pop twice and get **A** and **B** respectively. Calculate **B O A** and push it back to stack
 - When the expression is ended, the value in the stack is the final answer.

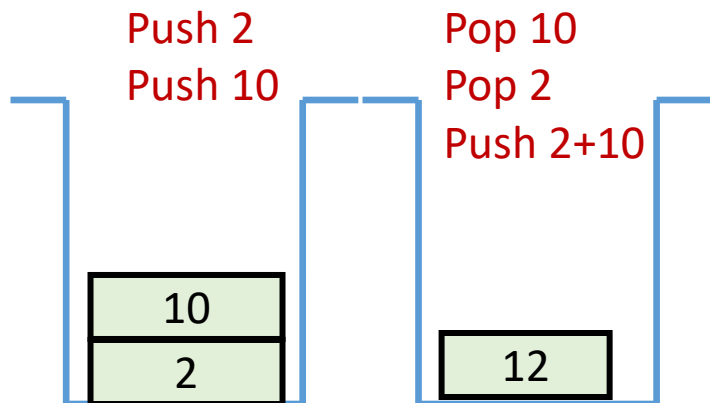
Expression Evaluation

- In postfix or reverse polish notation, every operator follows all of its operands
 - E.g., 2 10 + 9 6 - /
- Algorithm for evaluation of postfix expression
 - If the element is an operand, push it to stack
 - If the element is an operator $\circ$, pop twice and get A and B respectively. Calculate $B \circ A$ and push it back to stack
 - When the expression is ended, the value in the stack is the final answer.



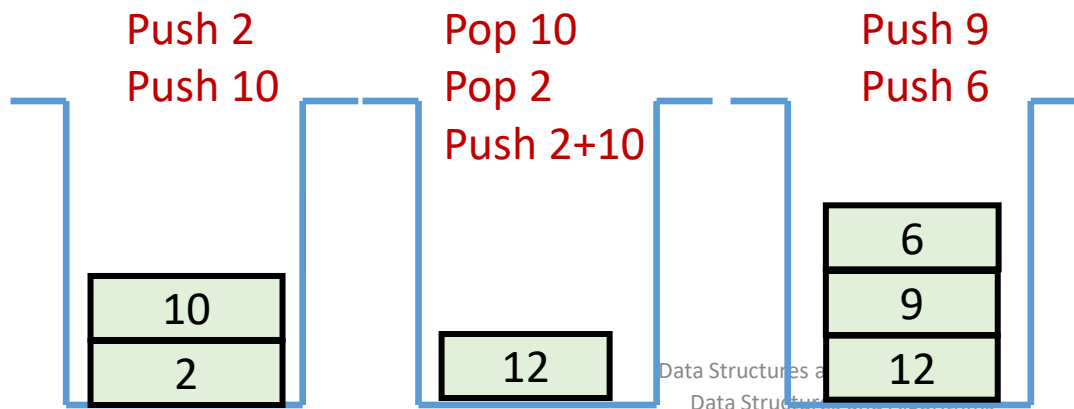
Expression Evaluation

- In postfix or reverse polish notation, every operator follows all of its operands
 - E.g., $2\ 10\ +\ 9\ 6\ -\ /$
- Algorithm for evaluation of postfix expression
 - If the element is an operand, push it to stack
 - If the element is an operator $\circ$, pop twice and get A and B respectively. Calculate $B\circ A$ and push it back to stack
 - When the expression is ended, the value in the stack is the final answer.



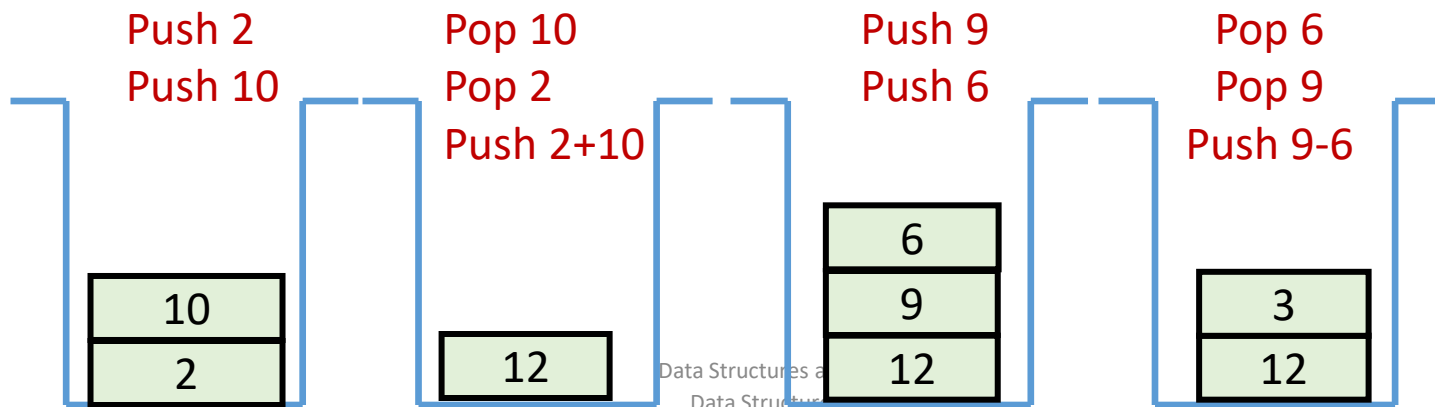
Expression Evaluation

- In postfix or reverse polish notation, every operator follows all of its operands
 - E.g., $2\ 10\ +\ 9\ 6\ -\ /$
- Algorithm for evaluation of postfix expression
 - If the element is an operand, push it to stack
 - If the element is an operator $\circ$, pop twice and get A and B respectively. Calculate $B\circ A$ and push it back to stack
 - When the expression is ended, the value in the stack is the final answer.



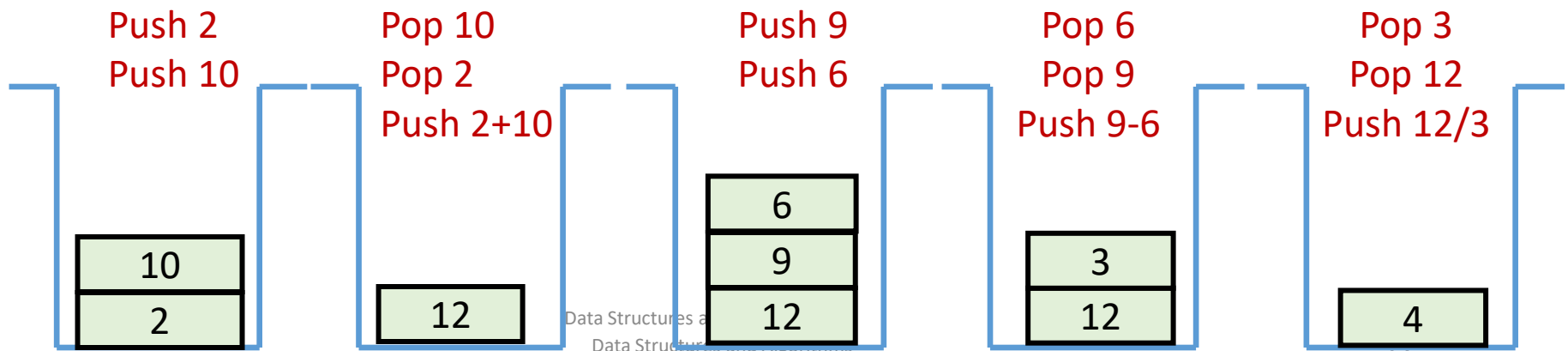
Expression Evaluation

- In postfix or reverse polish notation, every operator follows all of its operands
 - E.g., $2\ 10\ +\ 9\ 6\ -\ /$
- Algorithm for evaluation of postfix expression
 - If the element is an operand, push it to stack
 - If the element is an operator $\circ$, pop twice and get A and B respectively. Calculate $B\circ A$ and push it back to stack
 - When the expression is ended, the value in the stack is the final answer.



Expression Evaluation

- In postfix or reverse polish notation, every operator follows all of its operands
 - E.g., 2 10 + 9 6 - /
- Algorithm for evaluation of postfix expression
 - If the element is an operand, push it to stack
 - If the element is an operator $\circ$, pop twice and get A and B respectively. Calculate $B \circ A$ and push it back to stack
 - When the expression is ended, the value in the stack is the final answer.



Back Tracking

- Backtracking is used in algorithms in which there are steps along some path (state) from some starting point to some goal
 - Find your way through a maze.
 - Find a path from one point to another point in a graph.
 - Play a game in which there are moves to be made.
- In all of these cases, there are choices to be made among a number of options. We need some way to remember these decision points in case we want/need to come back and try the alternative
 - Consider the maze. At a point where a choice is made, we may discover that the choice leads to a dead-end. We want to retrace back to that decision point and then try the other (next) alternative.

Implementation of Stacks

- Recall the reason why we usually don't implement the list using array?

Topic		Array	Linked List
Efficiency	push		
	pop		
	Top		
space			

Stack Implementation

- Data
 - **maxTop**: the max index of top element of stack
 - **top**: the index of the top element of stack
 - **values**: point to an array which stores elements of stack
 - **values** can of any data type but we use **double** for demonstration

```
typedef struct{  
    double* values;  
    int top;  
    int maxTop;  
} Stack;
```

Stack Implementation

- Methods

```
bool CreateStack(Stack *stack, int size);  
bool IsEmpty(Stack* stack);  
bool IsFull(Stack* stack);  
bool Top(Stack* stack, double* x);  
bool Push(Stack* stack, double x);  
bool Pop(Stack* stack, double* x);  
void DisplayStack(Stack* stack);  
void DestroyStack(Stack** pstack);
```

Methods

- `bool CreateStack(Stack *stack, int size);`
 - Creates an empty stack whose capacity is *size*
- `bool IsEmpty(Stack* stack);`
 - Returns true if the stack is empty and false otherwise
- `bool IsFull(Stack* stack);`
 - Returns true if the stack is full and false otherwise
- `bool Top(Stack* stack, double* x);`
 - Returns true if the operation is successful and false otherwise
 - Passes the value of the top element to *x*

Methods

- `bool Push(Stack* stack, double x);`
 - Add a new element with value `x` to the top of the stack
 - Returns true if the operation is successful and false otherwise
- `bool Pop(Stack* stack, double* x);`
 - Remove an element from the top of the stack
 - Returns true if the operation is successful and false otherwise
 - Passes the value of the top element to `x`
- `void DisplayStack(Stack* stack);`
 - Display the elements in stack from top to bottom
- `void DestroyStack(Stack** pstack);`
 - Frees the memory occupied by the stack values and stack struct
 - Point the stack to NULL

CreateStack

```
#include <stdlib.h>

bool CreateStack(Stack *stack, int size){
    if (size <= 0)
        return false;

    stack->values = (double*)malloc(sizeof(double)*size);
    stack->top = -1;
    stack->maxTop = size - 1;
    return true;
}
```

Why?

Push

```
bool Push(Stack* stack, double x) {  
    if(IsFull(stack))  
        return false;  
    stack->values[++stack->top] = x;  
    return true;  
}
```

This explains.

Using stack

```
#include "stack.h"
int main() {
    Stack* stack = (Stack*)malloc(sizeof(Stack));
    double val;
    CreateStack(stack, 5);
    Push(stack, 5);
    Push(stack, 6.5);
    Push(stack, -3);
    Push(stack, -8);
    DisplayStack(stack);
    if(Top(stack, &val))
        printf("Top: %.2f\n", val);
    Pop(stack, &val);
    if(Top(stack, &val))
        printf("Top: %.2f\n", val);
    while(!IsEmpty(stack))
        Pop(stack, &val);
    DisplayStack(stack);
    Push(stack, 5)
    DestroyStack(&stack);
    DisplayStack(stack);}
```

Side Notes

- All these values evaluate to false in C:

0

NULL

false

Side Notes II

- A typical file structure for a C program with user defined data structures
 - `stack.h`
 - Declares all the [data types](#) and [functions](#)
 - No implementation
 - `stack.cpp` (`stack.c`)
 - Includes [stack.h](#)
 - Implements all the functions in `stack.h`
 - `main.cpp` (`main.c`)
 - Includes [stack.h](#)
 - Implements the main function